

目录

因果回环：基于时间滞后与块级转移统计的序列预测模型	1
1. 模型概述	1
2. 输入与符号	2
2.1 时间节点	2
2.2 信息向量	2
2.3 块	2
2.4 最大时间跨度	3
3. 时间节点的块计数表示	3
4. 块级全连接统计	4
4.1 时间节点之间的向量全连接	4
4.2 块到块连接数量	4
4.3 示例	5
5. 时间滞后连接表	5
6. 训练原理	6
6.1 批量训练	6
6.2 增量训练	7
7. 预测原理	7
7.1 单个历史节点的预测贡献	7
7.2 多时间节点累加	8
7.3 概率分布	8
8. 模型数据结构	9
8.1 块计数历史	9
8.2 时间滞后连接表	9
8.3 预测结果	9
9. Python 实现	9
9.1 RandomHyperplaneLSH	9
9.2 CausalLoopModel	10
10. 输入数据格式	10
11. 快速使用	11
12. 使用已经分好块的数据	12
13. 增量更新示例	13
14. 读取连接表	13
15. 外积等价性验证	14
16. 复杂度	14
16.1 显式向量全连接	15
16.2 块计数外积	15
17. 模型边界	15
18. 核心公式	15

因果回环：基于时间滞后与块级转移统计的序列预测模型

1. 模型概述

因果回环是一种面向多向量时间序列的离散状态预测模型。

输入数据由一组按时间排序的节点组成：

$$t_0, t_1, \dots, t_{n-1}$$

每个时间节点包含数量不定的 m 维信息向量。模型首先使用局部敏感哈希 (Locality-Sensitive Hashing, LSH) 将向量映射到有限数量的块, 然后统计不同时间滞后下各块之间的连接次数。预测时, 模型根据最近若干时间节点中的块分布, 查询历史连接表并累加得到下一个时间节点的块分布。

模型适用于以下类型的数据:

- 每个时间节点包含多条向量;
- 不同时间节点的向量数量可以不同;
- 向量之间具有相似性结构;
- 需要利用多个时间跨度上的历史共现关系预测下一时刻;
- 需要支持增量更新和较低成本的在线推断。

模型的核心结构由三部分组成:

1. **向量分块**: 将连续的 m 维向量映射到 l 个离散块;
 2. **时间滞后连接表**: 保存不同时间距离下的块到块连接次数;
 3. **历史累加预测**: 根据最近 $t_{\max}$ 个时间节点计算下一节点的块分数和概率分布。
-

2. 输入与符号

2.1 时间节点

时间序列共有 n 个已知节点:

$$T = \{t_0, t_1, \dots, t_{n-1}\}$$

预测目标是下一个时间节点:

$$t_n$$

2.2 信息向量

时间节点 t 包含 N_t 条 m 维向量:

$$X_t = \{x_{t,1}, x_{t,2}, \dots, x_{t,N_t}\}$$

其中:

$$x_{t,i} \in \mathbb{R}^m$$

不同时间节点的 N_t 可以不同。

2.3 块

LSH 分块函数记为:

$$h : \mathbb{R}^m \rightarrow \{0, 1, \dots, l-1\}$$

其中 l 是块数量。每条向量被映射到一个块：

$$h(x_{t,i}) = a$$

表示向量 $x_{t,i}$ 属于块 a 。

2.4 最大时间跨度

模型只统计不超过 $t_{\max}$ 的时间滞后：

$$d = 1, 2, \dots, t_{\max}$$

其中：

- $d = 1$ 表示相邻时间节点；
- $d = 2$ 表示相隔两个时间节点；
- $d = t_{\max}$ 表示模型使用的最大时间跨度。

3. 时间节点的块计数表示

一个时间节点中的全部向量经过 LSH 分块后，可以表示为长度为 l 的块计数向量：

$$c_t = \begin{bmatrix} c_t(0) \\ c_t(1) \\ \vdots \\ c_t(l-1) \end{bmatrix}$$

其中：

$$c_t(a) = \# \{x \in X_t : h(x) = a\}$$

表示时间节点 t 中落入块 a 的向量数量。

例如，共有 4 个块，某个时间节点的块分布为：

块编号	向量数量
0	2
1	0
2	3

块编号	向量数量
3	1

则该时间节点的块计数向量为：

$$c_t = \begin{bmatrix} 2 \\ 0 \\ 3 \\ 1 \end{bmatrix}$$

块计数向量保留了该时间节点中各类向量的数量信息，并作为训练和预测的基本状态表示。

4. 块级全连接统计

4.1 时间节点之间的向量全连接

对于两个时间节点 t 和 $t + d$ ，从前一个节点中的每条向量指向后一个节点中的每条向量。

若：

- t 中有 N_t 条向量；
- $t + d$ 中有 N_{t+d} 条向量；

则共有：

$$N_t N_{t+d}$$

条有向连接。

4.2 块到块连接数量

设：

- 时间节点 t 的块 a 中有 $c_t(a)$ 条向量；
- 时间节点 $t + d$ 的块 b 中有 $c_{t+d}(b)$ 条向量。

由于两个时间节点之间采用全连接，块 a 到块 b 的连接数为：

$$c_t(a) c_{t+d}(b)$$

因此，两个时间节点之间的完整块级连接矩阵可以通过块计数向量的外积得到：

$$C_{t,d} = c_t c_{t+d}^T$$

其中：

$$C_{t,d}(a,b) = c_t(a)c_{t+d}(b)$$

矩阵的行表示源块，列表示目标块。

4.3 示例

假设：

$$c_t = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, \quad c_{t+d} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$$

则：

$$c_t c_{t+d}^\top = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 4 & 0 \\ 1 & 2 & 0 \\ 3 & 6 & 0 \end{bmatrix}$$

其中第 3 行第 2 列的值为 6，表示源节点块 2 中的 3 条向量与目标节点块 1 中的 2 条向量之间形成了 6 条连接。

块计数外积直接得到块级连接数量，无需显式创建和保存每一条向量级边。

5. 时间滞后连接表

模型为每个时间滞后 d 保存一个 $l \times l$ 的整数矩阵：

$$M_d \in \mathbb{N}^{l \times l}$$

其中：

$$M_d(a,b)$$

表示训练序列中所有相隔 d 个时间节点的节点对里，从块 a 指向块 b 的累计连接数量。

连接表定义为：

$$M_d = \sum_{t=0}^{n-d-1} c_t c_{t+d}^\top$$

逐元素表示为：

$$M_d(a, b) = \sum_{t=0}^{n-d-1} c_t(a) c_{t+d}(b)$$

模型一共保存：

$$M_1, M_2, \dots, M_{t_{\max}}$$

共 $t_{\max}$ 个连接矩阵。

完整连接表的形状为：

(tmax, num_blocks, num_blocks)

其中程序数组的第 $d - 1$ 个矩阵对应数学公式中的 M_d 。

6. 训练原理

6.1 批量训练

训练过程包含两个阶段。

阶段一：生成块计数序列 对每个时间节点：

1. 读取节点中的全部向量；
2. 使用 LSH 将每条向量映射到块；
3. 统计得到块计数向量 c_t 。

最终得到：

$$c_0, c_1, \dots, c_{n-1}$$

阶段二：累计连接表 对于每个时间滞后：

$$d = 1, 2, \dots, t_{\max}$$

遍历所有合法的时间节点对 $(t, t + d)$ ，执行：

$$M_d \leftarrow M_d + c_t c_{t+d}^T$$

伪代码如下：

初始化 $M[1 \dots t_{\max}]$ ，每个元素都是 $l \times l$ 的零矩阵

for $t = 0 \dots n-1$:

$c[t]$ = 将时间节点 t 转换为块计数向量

```

for d = 1 ... tmax:
    for t = 0 ... n-d-1:
        M[d] += outer(c[t], c[t+d])

```

6.2 增量训练

当新的真实时间节点到来时，不需要重新扫描全部历史数据。

设新节点的块计数向量为：

$$c_{\text{new}}$$

对于每个可用时间滞后 d ，取距离新节点 d 步的历史节点 c_{old} ，更新：

$$M_d \leftarrow M_d + c_{\text{old}} c_{\text{new}}^T$$

然后将新节点加入历史序列。

这种方式适合流式数据和在线更新。

7. 预测原理

预测目标为下一个时间节点 t_n 。

模型使用最近最多 t_{max} 个历史节点：

$$t_{n-1}, t_{n-2}, \dots, t_{n-t_{\text{max}}}$$

7.1 单个历史节点的预测贡献

距离目标 d 步的历史节点为：

$$t_{n-d}$$

其块计数向量为：

$$c_{n-d}$$

该节点使用时间滞后连接表 M_d 。

对目标块的预测贡献为：

$$s_d = c_{n-d}^T M_d$$

其中第 b 个目标块的分数为：

$$s_d(b) = \sum_{a=0}^{l-1} c_{n-d}(a) M_d(a, b)$$

该计算过程等价于：

1. 找到历史节点中所有被激活的源块；
2. 查询每个源块在连接表中的整行数据；
3. 按该源块中的向量数量进行累加；
4. 得到所有目标块的预测贡献。

7.2 多时间节点累加

所有可用历史节点的贡献相加：

$$s = \sum_{d=1}^D c_{n-d}^T M_d$$

其中：

$$D = \min(t_{\max}, n)$$

最终得到长度为 l 的预测分数向量：

$$s = \begin{bmatrix} s(0) \\ s(1) \\ \vdots \\ s(l-1) \end{bmatrix}$$

7.3 概率分布

将预测分数归一化：

$$p(b) = \frac{s(b)}{\sum_{j=0}^{l-1} s(j)}$$

得到下一时间节点各块的预测概率分布：

$$p = \begin{bmatrix} p(0) \\ p(1) \\ \vdots \\ p(l-1) \end{bmatrix}$$

当所有预测分数均为 0 时，说明当前历史状态在连接表中没有可用记录。代码在这种情况下返回全 0 概率向量。

8. 模型数据结构

8.1 块计数历史

每个时间节点保存一个长度为 l 的整数向量：

```
count_history[t].shape == (num_blocks,)
```

8.2 时间滞后连接表

连接表保存为三维整数数组：

```
transition_tables.shape ==  
(tmax, num_blocks, num_blocks)
```

其中：

```
transition_tables[d - 1]
```

表示时间滞后 d 的连接表 M_d 。

8.3 预测结果

预测返回：

- scores: 各块的整数累计分数；
 - probabilities: 由分数直接归一化得到的浮点概率分布。
-

9. Python 实现

实现文件：

```
causal_loop.py
```

依赖：

```
pip install numpy
```

主要类包括：

9.1 RandomHyperplaneLSH

使用随机超平面生成二进制签名，并将签名映射到固定数量的块。

构造参数：

```
RandomHyperplaneLSH(
    num_blocks=64,
    num_hash_bits=None,
    random_state=42,
)
```

- num_blocks: 块数量;
- num_hash_bits: 随机超平面数量, 默认根据块数量自动计算;
- random_state: 随机种子, 用于保证分块结果可复现。

9.2 CausalLoopModel

负责块计数、连接表训练、增量更新和预测。

构造参数:

```
CausalLoopModel(
    num_blocks=64,
    tmax=5,
    hasher=hasher,
)
```

- num_blocks: 块数量 l ;
- tmax: 最大时间跨度;
- hasher: 向量分块器。

10. 输入数据格式

每个时间节点使用一个二维 NumPy 数组表示:

```
time_node = np.array(
    [
        [0.1, 0.2, 0.3],
        [0.4, 0.5, 0.6],
        [0.7, 0.8, 0.9],
    ],
    dtype=np.float64,
)
```

数组形状为:

(该节点的向量数量, 向量维度)

完整时间序列使用列表表示:

```
time_nodes = [
    time_node_0,
```

```
time_node_1,  
time_node_2,  
]
```

要求:

- 所有时间节点的向量维度必须相同;
 - 每个时间节点的向量数量可以不同;
 - 空时间节点可以使用形状为 (0, m) 的数组;
 - 输入不能包含 NaN 或无穷大。
-

11. 快速使用

```
import numpy as np  
  
from causal_loop import CausalLoopModel, RandomHyperplaneLSH  
  
time_nodes = [  
    np.array(  
        [  
            [1.0, 0.2],  
            [0.9, 0.1],  
        ],  
        dtype=np.float64,  
    ),  
    np.array(  
        [  
            [0.1, 1.0],  
            [0.2, 0.8],  
            [1.0, 0.0],  
        ],  
        dtype=np.float64,  
    ),  
    np.array(  
        [  
            [0.8, 0.3],  
            [0.0, 1.0],  
        ],  
        dtype=np.float64,  
    ),  
]
```

```
hasher = RandomHyperplaneLSH(  
    num_blocks=8,  
    random_state=42,  
)  
  
model = CausalLoopModel(  
    num_blocks=8,  
    tmax=2,  
    hasher=hasher,  
)  
  
model.fit(time_nodes)  
  
result = model.predict()  
  
print(" 预测分数: ")  
print(result.scores)  
  
print(" 预测概率: ")  
print(result.probabilities)
```

12. 使用已经分好块的数据

如果上游系统已经完成向量分块，可以直接使用块计数向量训练，不必再次执行 LSH。

例如共有 4 个块：

```
import numpy as np  
  
from causal_loop import CausalLoopModel  
  
count_vectors = [  
    np.array([2, 0, 1, 0], dtype=np.int64),  
    np.array([0, 3, 1, 0], dtype=np.int64),  
    np.array([1, 0, 2, 1], dtype=np.int64),  
]  
  
model = CausalLoopModel(  
    num_blocks=4,  
    tmax=2,  
)
```

```
model.fit_counts(count_vectors)

result = model.predict()

print(result.scores)
print(result.proBABILITIES)
```

13. 增量更新示例

训练完成后，当新的真实时间节点到达时：

```
new_vectors = np.array(
    [
        [0.7, 0.4],
        [0.3, 0.9],
        [0.9, 0.2],
    ],
    dtype=np.float64,
)

model.partial_fit_node(new_vectors)
```

更新后，可以继续预测再下一个时间节点：

```
next_result = model.predict()
```

如果新节点已经表示为块计数向量，可以使用：

```
new_counts = np.array(
    [0, 2, 1, 0],
    dtype=np.int64,
)

model.partial_fit_counts(new_counts)
```

14. 读取连接表

读取时间滞后 d 的连接表：

```
table_d1 = model.get_transition_table(1)
table_d2 = model.get_transition_table(2)
```

其中：

```
table_d1[a, b]
```

表示训练数据中，相隔 1 个时间节点时，从块 a 指向块 b 的累计连接数量。

15. 外积等价性验证

代码提供验证函数：

```
verify_outer_product_equivalence(  
    source_block_ids,  
    target_block_ids,  
    num_blocks,  
)
```

该函数分别使用：

1. 显式逐向量全连接；
2. 块计数外积；

计算同一对时间节点的块级连接矩阵，并检查结果是否完全一致。

示例：

```
import numpy as np  
  
from causal_loop import verify_outer_product_equivalence  
  
source_ids = np.array([0, 0, 2, 3])  
target_ids = np.array([1, 1, 2])  
  
is_equal = verify_outer_product_equivalence(  
    source_ids,  
    target_ids,  
    num_blocks=4,  
)  
  
print(is_equal)
```

输出：

```
True
```

16. 复杂度

设：

- 时间节点数为 n ；

- 每个时间节点平均有 r 条向量；
- 块数量为 l ；
- 最大时间跨度为 $t_{\max}$ 。

16.1 显式向量全连接

若逐条建立向量级连接，训练复杂度约为：

$$O(nt_{\max}r^2)$$

16.2 块计数外积

使用密集矩阵外积时，训练复杂度约为：

$$O(nt_{\max}l^2)$$

连接表的存储复杂度为：

$$O(t_{\max}l^2)$$

当每个时间节点包含大量向量，而块数量明显小于向量数量时，块计数外积能够显著降低计算量。

17. 模型边界

该实现基于以下定义：

- 每条向量只映射到一个块；
- 时间节点之间采用全连接；
- 所有向量级连接权重均为 1；
- 连接方向始终从较早时间节点指向较晚时间节点；
- 连接表保存块到块的累计整数数量；
- 预测使用最近最多 $t_{\max}$ 个时间节点；
- 各时间滞后的预测贡献直接相加；
- 概率由累计分数直接归一化得到。

如果连接权重需要由向量距离决定，或者只允许部分向量之间建立连接，则需要调整连接矩阵的计算方式。

18. 核心公式

训练：

$$M_d = \sum_{t=0}^{n-d-1} c_t c_{t+d}^\top$$

预测分数：

$$s = \sum_{d=1}^D c_{n-d}^\top M_d$$

其中：

$$D = \min(t_{\max}, n)$$

概率分布：

$$p(b) = \frac{s(b)}{\sum_j s(j)}$$

这三个公式分别对应模型的连接统计、历史查询和下一时间节点预测。